MARTINGALE DEEP NEURAL NETWORK FOR VERY HIGH-DIMENSIONAL STOCHASTIC OPTIMAL CONTROLS*

WEI CAI†, SHUIXIN FANG‡, WENZHONG ZHANG§, AND TAO ZHOU‡

Abstract. We propose a martingale deep learning method for very high-dimensional stochastic optimal control problems (SOCPs), based on their Hamilton–Jacobi–Bellman equations and verification theory. The method couples a martingale formulation for the value function with an integral optimality condition for the feedback control. We separate a pilot process for state-space exploration from control-dependent system processes for martingale enforcement, allowing the latter to be simulated parallelly across time steps. Extending martingale differencing to controlled dynamics yields loss functions for both the value function and the control that require no explicit evaluation of derivatives of the value network with respect to time and state. An adversarial Galerkin formulation avoids the direct computation of conditional expectations, while the integral control loss avoids pointwise minimization. Numerical experiments demonstrate accuracy and efficiency for SOCPs in dimensions up to 10,000.

Key words. Hamilton–Jacobi–Bellman equation, stochastic optimal control, deep learning, adversarial networks, martingale formulation.

MSC codes. 35K59, 35Q93, 49L20, 65M75, 65C05

1. Introduction.

Stochastic optimal control problems (SOCPs) concern the control of dynamical systems subject to random perturbations, with the goal of optimizing a prescribed performance criterion [54]. These problems arise widely in decision-making and systems under uncertainty, with applications in engineering, economics, biology, and rare event simulation [22,62]; see, e.g., [46,68,70]. Their analysis and numerical solution have long been central topics in applied mathematics.

The theory of SOCPs has evolved along two pioneering and complementary directions: Pontryagin's maximum principle [8, 11, 12, 41, 52], which provides necessary conditions for optimal controls via adjoint processes [67, p. 118], and Bellman's dynamic programming [5–7, 9], which characterizes optimal controls through stochastic verification theorems [79] and reduces the problem to solving the Hamilton–Jacobi–Bellman (HJB) equation [67, p. 163]. After decades of development, these two perspectives were connected with the theory of backward stochastic differential equations (BSDEs), a concept originally introduced by Bismut [10] as adjoint equations in the stochastic maximum principle (SMP), and later established as a systematic theory by Pardoux and Peng [51]. Nowadays, the theory of SOCPs continues to be an active area of research, with ongoing advances in analysis, probability, and in more general

*Date: October 19, 2026 (First version available on arXiv:2408.14395 as preprint on Aug. 26, 2024).

Funding: The work of WC is funded by Clements Chair of Applied Math. at SMU. The work of SF is supported by the Young Scientists Fund (Category C) of the National Natural Science Foundation of China (under grant 12601831), the Postdoctoral Fellowship Program of CPSF (under grant GZB20250705) and China Postdoctoral Science Foundation (under grant 2025M783150). The work of WZ is supported by the National Key R&D Program of China (under grant 2022YFA1005203) and the NSF of China (under grant 92270205, grant 1220012530). The work of TZ is supported by the NSF of China (under grant 12288201) and the Youth Innovation Promotion Association (CAS).

†Corresponding author. Department of Mathematics, Southern Methodist University, 208 Clements Hall, 3100 Dyer St, Dallas, TX 75205, USA. (cai@smu.edu).

‡Institute of Computational Mathematics and Scientific/Engineering Computing, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China. (sxfang@amss.ac.cn, tzhou@lsec.cc.ac.cn).

§Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou, China (wenzhong@ustc.edu.cn).

This manuscript is for review purposes only.

problem settings [13, 19, 31, 38, 53, 64].

Numerical methods for SOCPs play a central role in bridging theoretical developments and practical applications. However, designing efficient solvers remains highly challenging in high-dimensional settings, which frequently arise in applications involving large state and control spaces. It is well-known that conventional numerical methods [3, 14, 24, 25, 50, 57, 60, 61, 65, 73–75] suffer from the curse of dimensionality (CoD) [6]: their computational complexity grows exponentially with the problem dimension, making them impractical for high-dimensional SOCPs.

In recent years, the incorporation of deep learning into numerical methods has led to substantial progress in tackling the CoD, yielding practical solvers for high-dimensional SOCPs. Existing deep learning methods for SOCPs can be broadly categorized as follows: (a) SMP approach: the SMP characterizes the optimal control through a coupled forward-backward stochastic differential equations (FBSDEs) system together with a maximum condition [67, p. 118], and this system is then solved by deep learning; see, e.g., [16, 34, 36, 37]. (b) PDE approach: the dynamic programming principle reduces the SOCP to solving the associated HJB equation [67, p. 163], which is then tackled by deep-learning-based PDE solvers, e.g., [1, 4, 20, 21, 26, 56, 58, 63]. (c) Hybrid approach: SMP- and PDE-based approaches are combined to improve state-space exploration and learning efficiency; see, e.g., [44, 49]. (d) Reinforcement learning approach: the SOCP is reformulated as a Markov decision process and solved using policy-iteration methods; see, e.g., [2, 35, 39, 40, 76–78].

Despite the notable success of deep learning approaches, existing methods still face substantial challenges in very high-dimensional settings, for example when the state dimension d or control dimension m is on the order of thousands or more. These challenges arise mainly from the following four sources: (i) HJB equations involve Hessian matrices with $d \times d$ entries, and PDE- and hybrid-based approaches typically evaluate these Hessians through automatic differentiation of deep neural networks, which become prohibitively costly in computation and memory as the dimension increases. This bottleneck has motivated a line of recent works on high-dimensional PDEs [30, 32, 33, 42, 43, 59, 72]. (ii) SMP- and reinforcement-learning-based approaches avoid explicit Hessian evaluation through reformulation, but they require simulation of sample paths of the state process. These simulations are sequential in time and must be repeatedly updated as the control changes during training, which substantially limits the parallelizability of the algorithms. (iii) Unlike deterministic optimal control problems, general SOCPs feature a second-order maximum principle [67, Theorem 3.2, p. 118], in which the adjoint processes consist of a quadruple of dimensions d, $d \times d_0$, $d \times d$, and $d \times d \times d_0$, respectively, where d_0 is the dimension of the Brownian motion. Even in the simplest nontrivial case, there are still two adjoint processes, of dimensions d and $d \times d_0$; see [67, Cases 1 and 2, pp. 119–120]. Approximating these processes is quite challenging when d and d_0 are large. (iv) Both the SMP and HJB formulations involve an embedded optimization problem for the optimal control, whose minimizer generally does not admit a closed-form expression. As a result, pointwise optimization is required at every time-space point, which further exacerbates the CoD problem.

To address these challenges, we develop a new deep learning method for very high-dimensional SOCPs based on their associated HJB equations and optimal control verification theory. Our work consists of major algorithmic innovations as detailed below based on the mathematical formulations of the SOCPs in DeepMartNet [18] and SOC-MartNet [16], which utilize martingale approaches [15, 39, 40]. The key advances are the separation of pilot processes from control-dependent system processes

to enable temporal parallelism and the extension of martingale differencing techniques for parabolic PDEs [17,29,66] to controlled dynamics, yielding loss functions for both the value function and the control that require no explicit evaluation of derivatives of the value network. Specifically, the method combines the following components:

1. Following the approach of reinforcement learning, we split the HJB equation into two coupled subproblems: one for the value function and one for the optimal control. For the value-function subproblem, we extend DeepMartNet [15, 18] to reformulate the equation as a martingale problem, therefore avoiding all derivative computations in the original PDE.
2. To improve the parallelizability of the martingale formulation, we introduce two types of stochastic processes: a pilot process to explore the state space and system processes to encode the martingale property of the value-function subproblem. The pilot process is decoupled from training and can be generated offline. The system processes can be simulated independently at different time steps, enabling mini-batch sampling and parallel loss evaluation in both time and space.
3. Following the idea of SOC-MartNet [16], we enforce the martingale formulation with a Galerkin method, which is realized with adversarial learning techniques [69]. This avoids computation of conditional expectations in the martingale formulation, and further improves the efficiency of the method.
4. Following SOC-MartNet [16], we represent the feedback control by a neural network. Its loss is designed to minimize the integrated residual of the optimality condition over the state space explored by the pilot process. This approach eliminates the need for pointwise optimization at every time-space point.

Combining these ideas yields a martingale deep learning method for SOCPs and HJB equations. Numerical results show that the proposed method efficiently solves problems, including cases in which the optimal control does not admit an explicit expression, in dimensions up to 10^4.

The rest of this paper is organized as follows. Section 2 presents the setup of the SOCP and a brief review of the dynamic programming principle. Section 3 develops the martingale deep learning method for SOCPs and the associated HJB equations. Section 4 presents the numerical results. Some concluding remarks are given in section 5.

2. Problem setup and preliminaries.

2.1. Problem setup. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a complete probability space equipped with a filtration $(\mathcal{F}_t)_{t\in[0,T]}$ generated by the standard Brownian motion $B : [0,T] \times \Omega \to \mathbb{R}^q$, where $T > 0$ is a determined terminal time. The state equation is given by the following stochastic differential equation (SDE):

$$X_t = X_0 + \int_0^t \mu\big(s, X_s, u(s, X_s)\big)\,\mathrm{d}s + \int_0^t \sigma\big(s, X_s, u(s, X_s)\big)\,\mathrm{d}B_s, \quad t \in [0,T], \tag{2.1}$$

where $X_0 : \Omega \to \mathbb{R}^d$ is an $\mathcal{F}_0$-measurable random variable with a prescribed distribution; the stochastic integral with respect to B_s is understood in the Itô sense; $\mu : [0,T] \times \mathbb{R}^d \times U \to \mathbb{R}^d$ and $\sigma : [0,T] \times \mathbb{R}^d \times U \to \mathbb{R}^{d\times q}$ are given functions representing the drift and diffusion coefficients, respectively; $U \subset \mathbb{R}^m$ is a given control domain; $X : [0,T] \times \Omega \to \mathbb{R}^d$ is called the state process corresponding to the state-feedback

control $u : [0,T] \times \mathbb{R}^d \to U$, which belongs to the admissible set $\mathcal{U}_{\text{ad}}$ defined by

$$\mathcal{U}_{\text{ad}} := \left\{ u : [0,T] \times \mathbb{R}^d \to U : u \text{ is Borel measurable} \right\}. \tag{2.2}$$

Here the Borel measurability of u is required to ensure the validity of the relevant stochastic integrals and expectations in the subsequent analysis. To avoid technicalities, we assume that for any $u \in \mathcal{U}_{\text{ad}}$, the SDE (2.1) admits a unique strong solution X. Sufficient conditions for this assumption can be found in, e.g., [67, Chapter 1, Theorem 6.17].

The performance of a control $u \in \mathcal{U}_{\text{ad}}$ is measured by the cost functional

$$J(u) := \mathbb{E}\left[\int_0^T c\big(s, X_s, u(s, X_s)\big)\, \mathrm{d}s + g(X_T) \right], \tag{2.3}$$

where $c : [0,T] \times \mathbb{R}^d \times U \to \mathbb{R}$ and $g : \mathbb{R}^d \to \mathbb{R}$ are given functions representing the running cost and the terminal cost, respectively. The SOCP considered by this paper is to find an optimal control $u^* \in \mathcal{U}_{\text{ad}}$ minimizing the cost functional, i.e.,

$$J(u^*) = \inf_{u \in \mathcal{U}_{\text{ad}}} J(u). \tag{2.4}$$

2.2. Dynamic programming principle. To lay the theoretical foundation for our method, we briefly review the dynamic programming principle in stochastic control; see, e.g., [67, Chapter 4] and [54, Chapter 3] for comprehensive treatments.

The dynamic programming principle is based on the value function defined by

$$v(t,x) := \inf_{u \in \mathcal{U}_{\text{ad}}} \mathbb{E}\left[\int_t^T c\big(s, X_s, u(s, X_s)\big)\, \mathrm{d}s + g(X_T) \,\Big|\, X_t = x \right] \tag{2.5}$$

for $(t,x) \in [0,T] \times \mathbb{R}^d$, where X is the state process governed by (2.1) under control u. By [67, p. 182, Proposition 3.5; p. 190, Theorem 5.2], the value function v is the classical or viscosity solution to the following HJB equation:

$$\begin{cases} \partial_t v(t,x) + \inf\limits_{\kappa \in U} \left\{ \mathcal{L}^\kappa v(t,x) + c(t,x,\kappa) \right\} = 0, \quad (t,x) \in [0,T) \times \mathbb{R}^d, \\ v(T,x) = g(x), \quad x \in \mathbb{R}^d, \end{cases} \tag{2.6}$$

where $\mathcal{L}^\kappa$ is a differential operator defined by

$$\mathcal{L}^\kappa := \mu^\top(t,x,\kappa)\partial_x + \frac{1}{2}\,\mathrm{Tr}[\sigma\sigma^\top(t,x,\kappa)\,\partial^2_{xx}], \quad (t,x,\kappa) \in [0,T) \times \mathbb{R}^d \times U. \tag{2.7}$$

The operators $\partial_x := \nabla_x$ and $\partial^2_{xx} := \nabla_x \nabla_x^\top$ denote the gradient and Hessian, respectively, and $\mathrm{Tr}[\cdot]$ denotes the trace of a matrix.

Under the regularity condition $v \in C^{1,2}$, i.e., $v(t,x)$ is once continuously differentiable in $t \in [0,T)$ and twice continuously differentiable in $x \in \mathbb{R}^d$, the verification theorem [67, Chapter 5, Theorem 5.1] implies that the optimal control u^* can be given by the following feedback form:

$$u^*(t,x) = \arg\min_{\kappa \in U} \left\{ \mathcal{L}^\kappa v(t,x) + c\big(t,x,\kappa\big) \right\}, \quad (t,x) \in [0,T) \times \mathbb{R}^d. \tag{2.8}$$

Thus, the SOCP (2.4) reduces to solving the HJB equation (2.6) for v and then computing u^* from (2.8). This paper aims to develop an efficient implementation of this procedure for high-dimensional SOCPs.

3. Martingale deep learning. To proceed, we substitute (2.8) into (2.6), reformulating the HJB equation as the following two-step procedure for determining the value function v and the optimal control u:

$$\text{find } v \text{ s.t. } \big(\partial_t + \mathcal{L}^u\big)v(t,x) + c\big(t,x,u(t,x)\big) = 0, \tag{3.1}$$

$$\text{find } u \text{ s.t. } u(t,x) = \arg\min_{\kappa\in U}\left\{\mathcal{L}^\kappa v(t,x) + c\big(t,x,\kappa\big)\right\}, \tag{3.2}$$

where (t,x) ranges over $[0,T)\times\mathbb{R}^d$; v satisfies the terminal condition in (2.6) a priori; $\mathcal{L}^u$ is defined by (2.7) with $\kappa = u(t,x)$.

In the remainder of this section, we develop a deep learning method to solve (u,v) simultaneously from (3.1) and (3.2). The primary difficulty is to evaluate the derivatives in $\mathcal{L}^\kappa v$, which involves the Hessian $\partial^2_{xx}v$ with $d\times d$ entries. A second difficulty is the minimization in (3.2), which generally lacks a closed-form minimizer; a direct approach requires pointwise optimization at every (t,x). These two difficulties incur substantial computational costs in high dimensions, constituting a primary source of the CoD. We address them with a martingale approach that is derivative-free, highly parallel, and free of pointwise minimization.

3.1. Stochastic processes for martingale formulation. Let π_N be a uniform time partition of $[0,T]$ defined by

$$\pi_N : 0 = t_0 < t_1 < \cdots < t_N = T, \quad t_n := n\Delta t, \quad \Delta t := T/N \tag{3.3}$$

with N the number of time steps. Define the Brownian increments by

$$\Delta B_{n+1} := B_{t_{n+1}} - B_{t_n}, \quad n = 0,1,\cdots,N-1,$$

where the subscript $n+1$ in ΔB_{n+1} is used to indicate that it is $\mathcal{F}_{t_{n+1}}$-measurable.

Let u_α and v_θ denote neural networks parameterized by α and θ, respectively, used to approximate the optimal control u and the value function v as defined in (3.1) and (3.2). The following two types of stochastic processes are the key ingredients for our method:

1. **(Pilot process)** Let $\hat{X} : \pi_N\times\Omega\to\mathbb{R}^d$ be a discrete pilot process (indicated by $\hat{}$) used to explore the state space $\mathbb{R}^d$, generated by the following Euler–Maruyama scheme:

$$\hat{X}_{n+1} := \hat{X}_n + \hat{\mu}_n\Delta t + \hat{\sigma}_n\Delta B_{n+1}, \quad n = 0,1,\cdots,N-1, \tag{3.4}$$

where $\hat{\mu}_n$ and $\hat{\sigma}_n$ are the drift and diffusion coefficients evaluated with a pilot control $\hat{u}$ in place of u, i.e.,

$$\hat{\mu}_n := \mu\big(t_n,\hat{X}_n,\hat{u}(t_n,\hat{X}_n)\big), \quad \hat{\sigma}_n := \sigma\big(t_n,\hat{X}_n,\hat{u}(t_n,\hat{X}_n)\big). \tag{3.5}$$

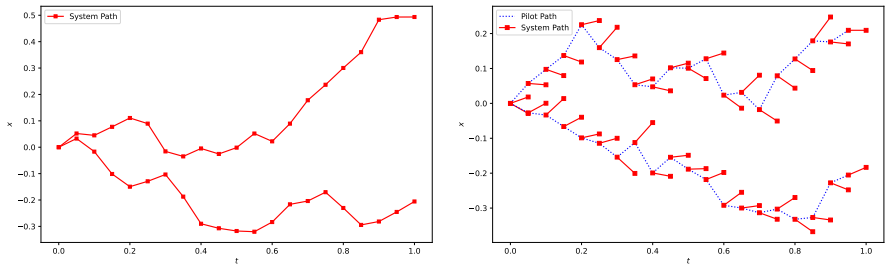
For parallelization purposes, the pilot path $(\hat{X}_n)_{n=0}^N$ need only be updated occasionally during training, with $\hat{u}$ taken as a frozen snapshot of u_α.

2. **(System process)** For $n = 0,1,\cdots,N-1$, define the system process as a random jump from $\hat{X}_n$ to a new location X_{n+1} at time t_{n+1} by

$$X_{n+1} := \hat{X}_n + \mu_n\Delta t + \sigma_n\Delta B_{n+1}, \quad n = 0,1,\cdots,N-1, \tag{3.6}$$

where μ_n and σ_n are the drift and diffusion coefficients evaluated using the current control network u_α, i.e.,

$$\mu_n := \mu\big(t_n,\hat{X}_n,u_\alpha(t_n,\hat{X}_n)\big), \quad \sigma_n := \sigma\big(t_n,\hat{X}_n,u_\alpha(t_n,\hat{X}_n)\big). \tag{3.7}$$



(a) Conventional approach (b) Our approach

FIG. 1. *Comparison of conventional and pilot–system path simulations. The horizontal and vertical axes represent time and state, respectively. In (b), red segments are system-path branches from the pilot path.*

The process $(X_{n+1})_{n=0}^{N-1}$ encodes the dynamics of $\mathcal{L}^{u_\alpha}$, which will be used to derive a martingale formulation for (3.1) in subsection 3.2.

The pilot and system processes have distinct computational features that align with their roles in the method. The pilot process $\hat{X}$ must be simulated sequentially in time; however, since its main purpose is to explore the state space, it can be generated offline and updated only occasionally during training. By contrast, the system process X depends on the current control network u_α and therefore must be updated at every training iteration to reflect the dynamics of $\mathcal{L}^{u_\alpha}$. Nevertheless, once the pilot path is generated, the system paths $(X_{n+1})_{n=0}^{N-1}$ can be computed independently across all time steps, which enables parallelization in the temporal direction. These differences are illustrated schematically in Figure 1.

Remark 3.1 (Controlled SDEs for space exploration). The pilot process $\hat{X}$ in (3.4) explores the state space by mimicking the controlled dynamics in (2.1). This strategy helps to focus the training on regions most relevant to the control problem. As shown in [44, 49], such an exploration strategy can significantly improve the performance of SOCP solvers, especially for problems with complex dynamics.

3.2. Martingale formulation for value function. We begin by deriving a martingale formulation for the value-function subproblem (3.1). The main idea is to parallelize the original DeepMartNet [15, 18] using the pilot and system processes introduced above.

The random jump from $\hat{X}_n$ to X_{n+1} in (3.6) can be interpreted as the evolution of the process $\hat{X}_{n,t} := \hat{X}_n + \mu_n(t-t_n) + \sigma_n(B_t - B_{t_n})$ as t increases from t_n to t_{n+1}. This evolution admits an infinitesimal generator $\partial_t + \mathcal{L}^{u_\alpha}$, where the coefficients in $\mathcal{L}^{u_\alpha}$ are evaluated at $(t,x) = (t_n, \hat{X}_n)$ and held fixed during the evolution from t_n to t_{n+1}. Then the Dynkin formula [48, Theorem 7.4.1] implies that

$$
\begin{aligned}
&\mathbb{E}[v_\theta(t_{n+1}, X_{n+1}) \,|\, \hat{X}_n] - v_\theta(t_n, \hat{X}_n) \\
&= \mathbb{E}\Big[\int_{t_n}^{t_{n+1}} \big(\partial_t + \mathcal{L}^{u_\alpha}\big|_{(t,x)=(t_n,\hat{X}_n)}\big) v_\theta(s, \hat{X}_{n,s})\, \mathrm{d}s \,\Big|\, \hat{X}_n \Big] \\
&= (\partial_t + \mathcal{L}^{u_\alpha}) v_\theta(t_n, \hat{X}_n) \Delta t + \mathcal{O}(\Delta t^2),
\end{aligned}
\tag{3.8}
$$

where the remainder term is justified provided that v_θ belongs to $C^{2,4}$ with derivatives of at most polynomial growth. This condition is not met by the ReLU networks used in the experiments, for which (3.8) therefore serves to motivate the discrete formulation

rather than to provide an error estimate.

To relate the above formula to the HJB equation (3.1), we add the running cost term $c(t,x,u_\alpha(t,x))\Delta t$, evaluated at $(t,x)=(t_n,\hat{X}_n)$, to both sides, which yields

$$\begin{aligned}&\mathbb{E}\big[\mathcal{M}(t_n,\hat{X}_n,X_{n+1};u_\alpha,v_\theta)\,\big|\,\hat{X}_n\big]\\&=\big\{(\partial_t+\mathcal{L}^{u_\alpha})v_\theta(t_n,\hat{X}_n)+c\big(t_n,\hat{X}_n,u_\alpha(t_n,\hat{X}_n)\big)\big\}\Delta t+\mathcal{O}(\Delta t^2),\end{aligned}\tag{3.9}$$

where $\mathcal{M}$ is a deterministic function defined by

$$\mathcal{M}\left(t,x,x';u,v\right):=v\left(t+\Delta t,x'\right)-v(t,x)+c(t,x,u(t,x))\Delta t\tag{3.10}$$

for $(t,x,x')\in[0,T-\Delta t]\times\mathbb{R}^d\times\mathbb{R}^d$. Neglecting the remainder term $\mathcal{O}(\Delta t^2)$, the second line represents exactly the residual of (3.1) evaluated at $(t_n,\hat{X}_n)$, multiplied by Δt. Therefore, the term in the first line should vanish, which leads to the following martingale formulation for (3.1): train v_θ to satisfy

$$\mathbb{E}\big[\mathcal{M}(t_n,\hat{X}_n,X_{n+1};u_\alpha,v_\theta)\,\big|\,\hat{X}_n\big]=0,\quad n=0,1,\cdots,N-1,\tag{3.11}$$

where X_{n+1} is defined in (3.6).

The formulation in (3.11) is a derivative-free, time-discrete approximation of the PDE (3.1), which does not involve any derivatives of v_θ. However, a direct enforcement of (3.11) with v_θ requires a two-level Monte Carlo procedure: first sampling the pilot path $\hat{X}$, and then, for each realized $\hat{X}_n$, approximating the conditional expectation in (3.11) by generating multiple samples of X_{n+1} conditioned on $\hat{X}_n$. This nested Monte Carlo approach is computationally intensive.

To address this issue, we adopt the idea in [16, 69] and enforce (3.11) in a weak sense using a Galerkin method. Let $\rho_\eta:[0,T]\times\mathbb{R}^d\to\mathbb{R}^r$ be a neural network parameterized by η, which serves as a test function in the Galerkin method. Then (3.11) can be enforced in the weak sense by requiring v_θ to satisfy for any η,

$$\sum_{n=0}^{N-1}\mathbb{E}\left[\mathcal{M}(t_n,\hat{X}_n,X_{n+1};u_\alpha,v_\theta)\rho_\eta(t_n,\hat{X}_n)\right]=0,\tag{3.12}$$

which is the time-discretized version of the following Galerkin method proposed in [16],

$$\int_0^{T-\Delta t}\mathbb{E}\left[\mathcal{M}(t,\hat{X}_t,X_{t+\Delta t};u_\alpha,v_\theta)\rho_\eta(t,\hat{X}_t)\right]\,\mathrm{d}t=0.\tag{3.13}$$

Here $\int_0^{T-\Delta t}\mathbb{E}\left[\,\cdot\,\right]\,\mathrm{d}t$ serves as the inner product in the Galerkin method, with the probability measure of $\hat{X}_t$ times the Lebesgue measure on $[0,T]$ acting as the time-space weight function. A notable feature of (3.12) is that the Galerkin method is imposed in both the space and the time dimensions. A sufficiently rich test function space defined on $[0,T]\times\mathbb{R}^d$ is necessary to enforce the martingale condition in (3.11) at every time step $n=0,1,\cdots,N-1$. Consequently, ρ_η takes both the time and space coordinates as inputs. Moreover, its output is a vector in $\mathbb{R}^r$ rather than a scalar, which improves the stability of the adversarial training; see subsection 3.4 for details.

The condition in (3.12) can be equivalently written as the following minimax problem:

$$\min_\theta\max_\eta\left|L(u_\alpha,v_\theta,\rho_\eta)\right|^2,\tag{3.14}$$

where the loss function L is defined by

$$L(u_\alpha, v_\theta, \rho_\eta) := \sum_{n=0}^{N-1} \mathbb{E}\left[\mathcal{M}(t_n, \hat{X}_n, X_{n+1}; u_\alpha, v_\theta)\rho_\eta(t_n, \hat{X}_n)\right]. \tag{3.15}$$

Here $\mathcal{M}$ is given by (3.10); $\hat{X}_n$ and X_{n+1} are given by (3.4) and (3.6), respectively. The minimax problem in (3.14) can be solved via the adversarial training procedure proposed in [69]; see subsection 3.4 for details.

Remark 3.2 (Efficiency gain from weak formulation). Compared with enforcing (3.11) directly, which would require a two-level Monte Carlo procedure, the weak formulation (3.14) reduces the loss evaluation in (3.15) to a single-level procedure. Specifically, since the expectation in (3.15) is taken with respect to the joint distribution of $(\hat{X}_n, X_{n+1})$, it can be approximated by generating independent identically distributed (i.i.d.) pairs of $(\hat{X}_n, X_{n+1})$, instead of generating multiple samples of X_{n+1} for each realized $\hat{X}_n$. This substantially lowers the computational cost. A mini-batch version of (3.15) in such a single-level manner is given in (3.28) for practical implementation.

Remark 3.3 (Trick of vector-valued test function). Recall that ρ_η is constructed as an $\mathbb{R}^r$-valued test function. Accordingly, the loss L defined in (3.15) is also $\mathbb{R}^r$-valued, and $|\cdot|$ in (3.14) denotes the Euclidean norm on $\mathbb{R}^r$. Compared with the scalar-valued case $r = 1$, choosing vector-valued test functions, e.g., $r \geq 600$, substantially improves the stability of the adversarial training for (3.14), as evidenced in the numerical results in [16, 17]. A specific structure of ρ_η is given in (3.24).

Remark 3.4 (Connection to derivative-free methods for PDEs). The equality in (3.8) is a key step of the proposed method, as it eliminates the differential operator $\mathcal{L}^{u_\alpha}$ through a martingale differencing argument. Similar ideas have also been explored in [17, 29, 66] for solving high-dimensional parabolic PDEs.

3.3. Integral formulation for feedback control. We next derive an integral form of (3.2) for learning the optimal feedback control u. The main idea is to use (3.9) to remove the differential operator $\mathcal{L}^u$ and transform the pointwise minimization in (3.2) into an integral minimization over the state explored by $\hat{X}$.

Note that (3.2) can be equivalently written as

$$u(t, x) = \arg\min_{\kappa \in U} \left\{(\partial_t + \mathcal{L}^\kappa)v(t, x) + c\left(t, x, \kappa\right)\right\}, \tag{3.16}$$

since the term $\partial_t v(t, x)$ is independent of κ and thus does not affect the minimizer; that is, we seek an optimal feedback control u under the current value function v. With $P_{\hat{X}_n}(\cdot)$ denoting the probability density function of $\hat{X}_n$, the minimization condition (3.16) implies

$$\begin{aligned} u =& \arg\min_{\bar{u} \in \mathcal{U}_{\text{ad}}} \sum_{n=0}^{N-1} \int_{\mathbb{R}^d} \left\{(\partial_t + \mathcal{L}^{\bar{u}})v(t_n, x) + c(t_n, x, \bar{u}(t_n, x))\right\} P_{\hat{X}_n}(x)\Delta t \, \mathrm{d}x \\ =& \arg\min_{\bar{u} \in \mathcal{U}_{\text{ad}}} \sum_{n=0}^{N-1} \mathbb{E}\left[(\partial_t + \mathcal{L}^{\bar{u}})v(t_n, \hat{X}_n) + c(t_n, \hat{X}_n, \bar{u}(t_n, \hat{X}_n))\right]\Delta t, \end{aligned} \tag{3.17}$$

where the second line is obtained by expressing the integral in the first line as an expectation with respect to $\hat{X}_n$.

The integral minimization in (3.17) is clearly necessary for the pointwise minimization in (3.16). Perhaps less obviously, it is also sufficient under mild conditions. The key insight is that $\mathcal{U}_{\rm ad}$ consists of feedback controls, which allows $\bar{u} \in \mathcal{U}_{\rm ad}$ in (3.17) to take different values $\bar{u}(t,x)$ at different (t,x). Consequently, an integral minimizer must be pointwise optimal at almost every (t,x) under the law of $(\hat{X}_n)_{n=0}^{N-1}$. The following lemma, which extends [16, Lemma 3.2], rigorously establishes this sufficiency property.

LEMMA 3.5 (Sufficiency of integral minimization). *Let $(E,\mathcal{E},\nu)$ be a measure space, and equip $U \subset \mathbb{R}^m$ with the Borel σ-algebra $\mathcal{B}(U) := \{F \cap U : F \in \mathcal{B}(\mathbb{R}^m)\}$. Let $G : E \times U \to \mathbb{R}$ be $\mathcal{E} \otimes \mathcal{B}(U)$-measurable. Denote by $\mathcal{A}$ the set of measurable maps $a : E \to U$ such that $\int_E |G(z,a(z))|\,\mathrm{d}\nu(z) < \infty$. Assume that there exists $a^* \in \mathcal{A}$ such that $G(z,a^*(z)) = \inf_{\kappa\in U} G(z,\kappa)$ for ν-almost every $z \in E$. Then for every $\bar{a} \in \mathcal{A}$, the following implication holds:*

$$\begin{aligned} &\bar{a} \in \arg\min_{a\in\mathcal{A}} \int_E G(z,a(z))\,\mathrm{d}\nu(z) \\ \implies\quad &\bar{a}(z) \in \arg\min_{\kappa\in U} G(z,\kappa) \text{ for } \nu\text{-almost every } z \in E. \end{aligned} \tag{3.18}$$

Proof. Let $\bar{a}$ satisfy the first line of (3.18). Then

$$0 \le \int_E \big\{G(z,\bar{a}(z)) - G(z,a^*(z))\big\}\,\mathrm{d}\nu(z) \le 0, \tag{3.19}$$

where the first inequality follows from the definition of a^*, which gives $G(z,\bar{a}(z)) - G(z,a^*(z)) = G(z,\bar{a}(z)) - \inf_{\kappa\in U} G(z,\kappa) \ge 0$ for ν-almost every $z \in E$, and the second inequality follows from the first line of (3.18). Since the integrand in (3.19) is nonnegative ν-almost everywhere and has vanishing integral, it must vanish ν-almost everywhere. This proves the second line of (3.18). ∎

By Lemma 3.5 with $E = \{0,1,\dots,N-1\} \times \mathbb{R}^d$, $G = (\partial_t + \mathcal{L}^\kappa)v + c$ and ν the occupation measure of $(\hat{X}_n)_{n=0}^{N-1}$, the integral minimization in (3.17) implies the pointwise minimization in (3.16) in the sense that

$$u(t_n,\hat{X}_n) \in \arg\min_{\kappa\in U} \big\{(\partial_t + \mathcal{L}^\kappa)v(t_n,\hat{X}_n) + c(t_n,\hat{X}_n,\kappa)\big\}, \quad n = 0,1,\cdots,N-1,$$

almost surely. Consequently, the minimizer of (3.17) yields a feedback control u that is pointwise optimal, in a probabilistic sense, for the given value function v.

By the second line of (3.17) with (u,v) replaced by the neural networks (u_α, v_θ), and then using (3.9) to express the residual term in the second line of (3.17), we obtain the following integral formulation for learning the optimal feedback control:

$$u_\alpha = \arg\min_{\bar{u}\in\mathcal{U}_{\rm ad}} \sum_{n=0}^{N-1} \mathbb{E}\left[\mathcal{M}(t_n,\hat{X}_n,X_{n+1};\bar{u},v_\theta)\right], \tag{3.20}$$

where we have neglected the remainder term $\mathcal{O}(\Delta t^2)$ in (3.9). Using the loss function L defined in (3.15), and constructing u_α so that its values lie in U as described in (3.22), the minimization problem (3.20) can be solved by training u_α via

$$\min_\alpha L(u_\alpha, v_\theta, 1), \tag{3.21}$$

where $L(u_\alpha, v_\theta, 1)$ is given by (3.15) with ρ_η replaced by the constant function equal to 1. As shown in (3.21), the integral minimization in (3.20) eliminates the time-state space point-wise minimization in (3.16) and reduces the problem to a standard neural network training task.

By alternating the training of v_θ via (3.14) and u_α via (3.21), we obtain a unified martingale formulation for solving the SOCP (2.4). This formulation is simple, since both subproblems are based on the same loss function L in (3.15), differing only in the choice of test function. This unified structure simplifies implementation and improves computational efficiency by allowing intermediate quantities to be shared across the two subproblems.

3.4. Network architecture. Now we specify the architectures of the networks u_α, v_θ, and ρ_η used in the training procedures (3.14) and (3.21). To avoid technical complications, we assume that u_α is constructed such that its output always lies within the control domain U. This can be achieved by applying an appropriate output activation function to enforce the range, or, if U is convex, by projecting the output of a neural network onto U:

$$u_\alpha(t,x) = \operatorname*{arg\,min}_{y\in U} \big|\psi_\alpha(t,x) - y\big|, \quad (t,x) \in [0,T) \times \mathbb{R}^d, \tag{3.22}$$

where ψ_α is a neural network with parameters α. For regular U, such as box constraints of the form $U = \prod_{i=1}^m [a_i, b_i]$ with $-\infty < a_i \le b_i < +\infty$, this projection can be computed efficiently.

For the network v_θ, we assume that the terminal condition in (2.6) is satisfied by construction. This can be achieved by embedding the terminal condition $v(T,x) = g(x)$ directly into the architecture of v_θ, e.g.,

$$v_\theta(t,x) = \begin{cases} g(x), & \text{if } t = T, \\ \phi_\theta(t,x), & \text{if } t < T \end{cases} \tag{3.23}$$

for $(t,x) \in [0,T] \times \mathbb{R}^d$, where $\phi_\theta : [0,T) \times \mathbb{R}^d \to \mathbb{R}$ is a neural network parameterized by θ. The discontinuity of v_θ at $t = T$ does not cause any issues in training, since the network is only evaluated at the discrete time steps in π_N.

The adversarial network ρ_η in (3.15) plays the role of test functions in classical Galerkin methods for solving PDEs. By our experimental results, ρ_η is not necessarily very deep; instead, it can be a shallow network with enough output dimensionality. Following the multiscale neural network ideas in [45], we consider a typical ρ_η given by

$$\rho_\eta(t,x) = \sin\left(\Lambda\left(W_1 t + W_2 x + b\right)\right) \in \mathbb{R}^r, \tag{3.24}$$

where $\eta := (W_1, W_2, b) \in \mathbb{R}^r \times \mathbb{R}^{r\times d} \times \mathbb{R}^r$ is the trainable parameter; $\Lambda(\cdot)$ is a scale layer defined by

$$\Lambda(y_1, y_2, \cdots, y_r) = (c_1 y_1, c_2 y_2, \cdots, c_r y_r)^\top \in \mathbb{R}^r, \quad c_i := 1 + \frac{i-1}{r-1}(c-1) \tag{3.25}$$

for $y_i \in \mathbb{R}$, where $c > 0$ is a non-trainable hyperparameter; $\sin(\cdot)$ is the activation function applied to the outputs of Λ in an element-wise manner.

3.5. Learning algorithm. The learning algorithm is obtained by approximating the objectives in (3.14) and (3.21) with mini-batch estimates of the expectation in (3.15). Specifically, for each $n = 0, 1, \cdots, N-1$, let $\{(\hat{X}_n^m, X_{n+1}^m)\}_{m=1}^M$ be i.i.d. samples of $(\hat{X}_n, X_{n+1})$ generated by (3.4) and (3.6). Then, by the definition of L in (3.15), the objectives in (3.14) and (3.21) can be approximated by the following unbiased mini-batch estimators:

$$|L(u_\alpha, v_\theta, \rho_\eta)|^2 \approx L^\top(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_1) L(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_2), \tag{3.26}$$

$$L(u_\alpha, v_\theta, \rho_\eta) \approx L(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_1 \cup \mathbb{A}_2), \tag{3.27}$$

$$L(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_i) := \frac{1}{|\mathbb{A}_i|} \sum_{m \in \mathbb{A}_i} \sum_{n=0}^{N-1} \mathcal{M}(t_n, \hat{X}_n^m, X_{n+1}^m; u_\alpha, v_\theta) \rho_\eta(t_n, \hat{X}_n^m), \tag{3.28}$$

where $|\mathbb{A}_i|$ denotes the cardinality of the index sets $\mathbb{A}_i$ for $i = 1, 2$, sampled from $\{1, 2, \cdots, M\}$, such that $\mathbb{A}_1 \cap \mathbb{A}_2 = \emptyset$. The disjointness of $\mathbb{A}_1$ and $\mathbb{A}_2$ ensures that the mini-batch estimator in (3.26) is unbiased.

In summary, to solve (u, v) from (3.1) and (3.2), we train (u_α, v_θ) by enforcing (3.14) and (3.21). Specifically, we update v_θ and u_α by gradient descent to minimize the objectives in (3.14) and (3.21), respectively, and update the test network ρ_η by gradient ascent to maximize (3.14). In implementation, the objectives and their stochastic gradients are approximated using the mini-batch estimators in (3.26)–(3.27). For efficiency, the pilot paths $\hat{X}_n^m$ are updated only occasionally during training. Algorithm 3.1 summarizes the complete training procedure.

Remark 3.6 (Extension to quasilinear parabolic PDEs). Observe that the subproblem (3.1) reduces to a quasilinear parabolic PDE of v when the control u is replaced by the solution v itself; namely,

$$\big(\partial_t + \mathcal{L}^v\big) v(t,x) + c\big(t, x, v(t,x)\big) = 0, \quad (t,x) \in [0,T) \times \mathbb{R}^d, \tag{3.29}$$

with terminal condition the same as in (2.6). This observation shows that Algorithm 3.1 extends naturally to the quasilinear parabolic PDE (3.29). In practice, one removes Line 8 and replaces each remaining occurrence of u_α with v_θ. The resulting output v_θ then provides an approximation of the solution v to (3.29).

3.6. Discussions on advantages. The martingale deep learning method presented in Algorithm 3.1 builds upon our earlier work on DeepMartNet [15, 18] and SOC-MartNet [16], developed for solving PDEs and SOCPs, respectively. The differences between the proposed method and existing works are summarized as follows:

- Compared with DeepMartNet [15, 18] and other SDE-based methods such as [2, 34–37, 39, 40, 76–78], the proposed method avoids the simulation of full trajectories at every training iteration, as illustrated in Figure 1(a). In those existing methods, path simulation is sequential in time. Moreover, because the simulated paths depend on the control neural networks being trained, they must be regenerated whenever the parameters of the networks are updated. This online and temporally sequential path simulation substantially limits the computational efficiency, thus limiting the capability of these methods for very high-dimensional problems.
- Compared with the original SOC-MartNet [16], the method proposed in this paper avoids the need for derivative-dependent loss evaluation, as shown in (3.28). Although SOC-MartNet already avoids online path simulation, it

Algorithm 3.1 Martingale deep learning for the SOCP (2.4)

Input: $v_\theta/u_\alpha/\rho_\eta$: neural networks parameterized by $\theta/\alpha/\eta$; $\delta_0/\delta_1/\delta_2$: learning rates for the networks $v_\theta/u_\alpha/\rho_\eta$; I: maximum number of iterations; J/K: number of $(\theta,\alpha)/\eta$ updates per iteration; I_0 and $r\%$: interval for updating spatial samples and the percentage of samples updated; M: total number of sample paths retained in memory.

Initialize the networks u_α, v_θ and ρ_η
Generate the pilot paths $\{\hat{X}_n^m\}_{n=0}^N$, $1 \le m \le M$, by (3.4) with $\hat{u} = u_\alpha$.
for $i = 0, 1, \cdots, I-1$ **do**
    Sample disjoint index subsets $\mathbb{A}_1, \mathbb{A}_2$ from $\{1, 2, \cdots, M\}$.
    Generate the system paths $\{X_{n+1}^m\}_{n=0}^{N-1}$ for $m \in \mathbb{A}_1 \cup \mathbb{A}_2$ by (3.6).
    for $j = 0, 1, \cdots, J-1$ **do** // L is given by (3.28)
        $\theta \leftarrow \theta - \delta_0 \nabla_\theta \left\{ L^\top(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_1) L(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_2) \right\}$
        $\alpha \leftarrow \alpha - \delta_1 \nabla_\alpha L(u_\alpha, v_\theta, 1; \mathbb{A}_1 \cup \mathbb{A}_2)$
    end for
    for $k = 0, 1, \cdots, K-1$ **do**
        $\eta \leftarrow \eta + \delta_2 \nabla_\eta \left\{ L^\top(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_1) L(u_\alpha, v_\theta, \rho_\eta; \mathbb{A}_2) \right\}$
    end for
    if i is divisible by I_0 **then**
        Update $r\%$ of the pilot paths $\{\hat{X}_n^m\}_{n=0}^N$ by (3.4) with $\hat{u} = u_\alpha$
    end if
end for

Output: (v_θ, u_α) as approximations of (v, u).

still relies on automatic differentiation to evaluate the derivatives of v_θ. The proposed method instead introduces the random jumps $\hat{X}_n \mapsto X_{n+1}$ in (3.6) to represent the dynamics of $\mathcal{L}^{u_\alpha} v_\theta$, thereby eliminating the need for explicit derivative evaluations.

- Compared with PDE solvers, e.g., [4, 23, 26–28, 47, 55, 56, 63, 71], applied directly to HJB equations, our method avoids time and state-space pointwise minimization over the control variable when learning the optimal feedback control. Instead, the control is learned via the integral formulation in (3.20), similar to standard neural network training by loss minimization. This avoids the complex optimization steps required in SMP and HJB formulations.

In summary, the proposed method eliminates the need for derivative-dependent loss evaluation, online sequential path simulation, and pointwise minimization over the control at each time–state point. These features yield substantial improvements in computational efficiency, especially for very high-dimensional problems, as demonstrated in section 4.

4. Numerical examples. We will demonstrate the effectiveness of the proposed method by applying Algorithm 3.1 to a series of benchmark problems in high-dimensional settings. When presenting these results, the abbreviations "RE", "SD", "RT", "vs", and "Iter." are used for "relative error", "standard deviation", "runtime", "versus", and "Iteration", respectively. Here, RT denotes the wall-clock time required to solve the problem, excluding the time spent evaluating and logging the error indicators; multiplying RT by the number of GPUs used yields the corresponding GPU time.

Unless stated otherwise, the initial state X_0 in (2.1) is assumed to be uniformly

distributed on a set $D_0 \subset \mathbb{R}^d$, with $D_0 = S_0$ in subsection 4.2 and $D_0 = S_0 \cup S_1$ in the remaining subsections. This set is specified through two curves S_0 and S_1 in $\mathbb{R}^d$, defined as

$$S_0 := \{s\mathbf{1}_d : s \in [-1,1]\}, \quad \mathbf{1}_d := (1,1,\cdots,1)^\top \in \mathbb{R}^d. \tag{4.1}$$

$$S_1 := \{\boldsymbol{l}(s) : s \in [-1,1]\}, \quad \boldsymbol{l}(s) := \big(l_1(s), l_2(s), \cdots, l_d(s)\big)^\top \in \mathbb{R}^d \tag{4.2}$$

where $l_i(s) := s\,\mathrm{sgn}\,(\sin(i)) + \cos(i + \pi s)$ and $\mathrm{sgn}(z) := -1, 0, 1$ for $z < 0$, $z = 0$, and $z > 0$, respectively. Here S_0 is a line segment along the main diagonal of the hypercube $[-1,1]^d$, whereas S_1 is a non-diagonal curve whose coordinates oscillate with the coordinate index i.

We introduce the following indicators to quantify performance of the proposed method:

$$\mathrm{RE}_v^{t_0} := \Big\{ \sum_{x \in \bar{D}_0} |v_\theta(0,x) - v(0,x)| \Big\} / \Big\{ \sum_{x \in \bar{D}_0} |v(0,x)| \Big\}, \tag{4.3}$$

$$\mathrm{RE}_v^{\mathrm{path}} := \Big\{ \sum_{n=0}^{N-1} \sum_{m=1}^{M_{\mathrm{test}}} \big|v_\theta(t_n, \bar{X}_n^m) - v(t_n, \bar{X}_n^m)\big| \Big\} / \Big\{ \sum_{n=0}^{N-1} \sum_{m=1}^{M_{\mathrm{test}}} \big|v(t_n, \bar{X}_n^m)\big| \Big\}, \tag{4.4}$$

$$\mathrm{RE}_{\mathrm{c}} := \frac{|\hat{J}(u_\alpha) - v(0,x_0)|}{|v(0,x_0)|}, \quad \hat{J}(u_\alpha) := \frac{1}{M_{\mathrm{test}}} \sum_{m=1}^{M_{\mathrm{test}}} \Big\{ \sum_{n=0}^{N-1} c_n^m \Delta t + g(\bar{X}_N^m) \Big\}. \tag{4.5}$$

where $c_n^m := c(t_n, \bar{X}_n^m, u_\alpha(t_n, \bar{X}_n^m))$, and the paths $\bar{X}_n^m$ are generated by simulating the discretized state equation (3.4) with $\hat{u} = u_\alpha$ and $\bar{X}_0^m = x_0 := \mathbf{0}_d \in \mathbb{R}^d$ with M_{test} independent realizations. The indicator $\mathrm{RE}_u^{\mathrm{path}}$ is defined analogously to $\mathrm{RE}_v^{\mathrm{path}}$, with v_θ replaced by u_α and v by u. The set $\bar{D}_0$ is a discretized version of D_0: $\bar{D}_0 := \bar{S}_0$ in subsection 4.2 and $\bar{D}_0 := \bar{S}_0 \cup \bar{S}_1$ in the remaining subsections, where, for $i = 0, 1$, $\bar{S}_i$ denotes a set of n_i points randomly sampled from S_i and then fixed throughout the training process. Specifically, we set $n_i = 300$ in subsection 4.2 and $n_i = 100$ in the remaining subsections.

When exact solutions are unavailable, we use the reference solutions specified for the corresponding numerical examples in place of the exact value function v and optimal control u. Depending on the computational cost of obtaining the reference solutions, M_{test} is set to 32,768 for the HJB equations in subsection 4.2 and to 4,096 for those in subsections 4.3 and 4.4. The only exceptions are the pathwise errors in subsection 4.2, which use 128 paths because reference solutions along the simulated paths are prohibitively expensive.

The above indicators can be interpreted as follows. The indicator $\mathrm{RE}_v^{t_0}$ quantifies the accuracy of the learned value function $v_\theta(t,x)$ at $t = 0$ for $x \in D_0$. The curves S_0 and S_1 are used to better visualize the solution profile in high dimensions. The indicators $\mathrm{RE}_v^{\mathrm{path}}$ and $\mathrm{RE}_u^{\mathrm{path}}$ quantify the accuracy of v_θ and u_α, respectively, along the simulated paths $\bar{X}_n^m$. They thus assess the accuracy of our method in regions of the high-dimensional state space explored by the dynamics. The indicator RE_{c} denotes the relative error of the cost functional $J(u_\alpha)$ in (2.3), evaluated at the learned control u_α. Ideally, if u_α is the optimal control and there is no Monte Carlo error, RE_{c} is zero; thus, it measures the optimality of the learned control u_α.

4.1. Parameter settings. The following parameter settings serve as the default choices for Algorithm 3.1 in the numerical experiments. For specific examples, certain parameters are adjusted either to ensure fair comparisons with existing methods or

TABLE 1
Results of Algorithm 3.1 *for HJB-1, -2, and -3 in subsection* 4.2. *Mean and SD denote the mean and standard deviation of each reported value over five independent runs. Device:* 8×*A100.*

Equation	d	n_{unit}	n_{layer}	$\text{RE}_v^{t_0}$		$\text{RE}_v^{\text{path}}$		RE_{c}		RT (s)
				Mean	SD	Mean	SD	Mean	SD	
HJB-1	10,000	$d+10$	4	2.97E-3	1.8E-4	1.02E-2	7.0E-4	1.76E-3	3.2E-5	1719
HJB-2	10,000	$d+10$	4	2.17E-2	6.8E-3	6.71E-3	3.2E-3	8.17E-3	3.2E-4	1720
HJB-3	10,000	$d+10$	6	3.95E-2	4.8E-3	2.03E-2	4.2E-3	1.35E-3	2.5E-5	2417
HJB-3	2,000	$d+10$	4	1.21E-1	1.0E-2	5.55E-2	3.3E-3	6.14E-2	2.6E-3	285
HJB-3	2,000	$5d+10$	4	3.48E-2	6.1E-3	3.31E-2	2.7E-3	3.07E-3	9.4E-5	1037

to test the scalability of the proposed method; the corresponding values are reported in the respective subsections.

The neural networks u_α and v_θ are constructed as in (3.22) and (3.23), respectively. Their inner networks ψ_α and ϕ_θ are fully connected with $n_{\text{layer}} = 4$ hidden layers, each containing n_{unit} ReLU units; the values of n_{unit} used in the experiments are reported together with the corresponding numerical results. The adversarial network ρ_η is taken as in (3.24), with output dimension $r = 600$ and scaling parameter $c = 100$, except for the example in subsection 4.3, which uses $c = 10$.

For parameter settings of Algorithm 3.1, the total number M of sample paths retained in memory is set to $M = 10^4$ for all the tests, except for the scalar-channel example in subsection 4.4, which uses $M = 10{,}240$. The number of time steps in (3.3) is set to $N = 32$ unless otherwise specified; the comparison with SOC-MartNet [16] in subsection 4.2 adopts $N = 100$ to align the time grid with that of SOC-MartNet. The mini-batch sizes $|\mathbb{A}_1| = |\mathbb{A}_2|$ are set to 200, except for the example in subsection 4.3, which uses $|\mathbb{A}_1| = |\mathbb{A}_2| = 400$. The interval for updating pilot paths and the percentage of paths updated are set to $I_0 = M/|\mathbb{A}_1 \cup \mathbb{A}_2|$ and $r\% = 20\%$, respectively. The inner iteration steps are $J = 2K = 2$. The stochastic gradient updates are performed by the RMSProp algorithm. At the i-th iteration step, the learning rates of u_α and v_θ are both set to $\delta_0 \times 0.5 \times 10^{-3} \times 0.01^{i/I}$, and the learning rate of ρ_η is set to $\delta_0 \times 10^{-2} \times 0.01^{i/I}$, where $\delta_0 := 3d^{-0.5}$ for $d \le 1000$ and $\delta_0 := 3d^{-0.8}$ for $d > 1000$.

All experiments are implemented in Python 3.12 using PyTorch 2.11. Automatic mixed precision is employed in the forward and backward passes of the neural networks, except in the comparison with SOC-MartNet [16] in subsection 4.2, where automatic mixed precision is disabled so as to match the precision used in SOC-MartNet. The algorithm is accelerated by the strategy of Distributed Data Parallel (DDP) [1] on compute nodes equipped with NVIDIA A100-SXM4-80GB GPUs, with the number of GPUs per run reported together with the corresponding results.

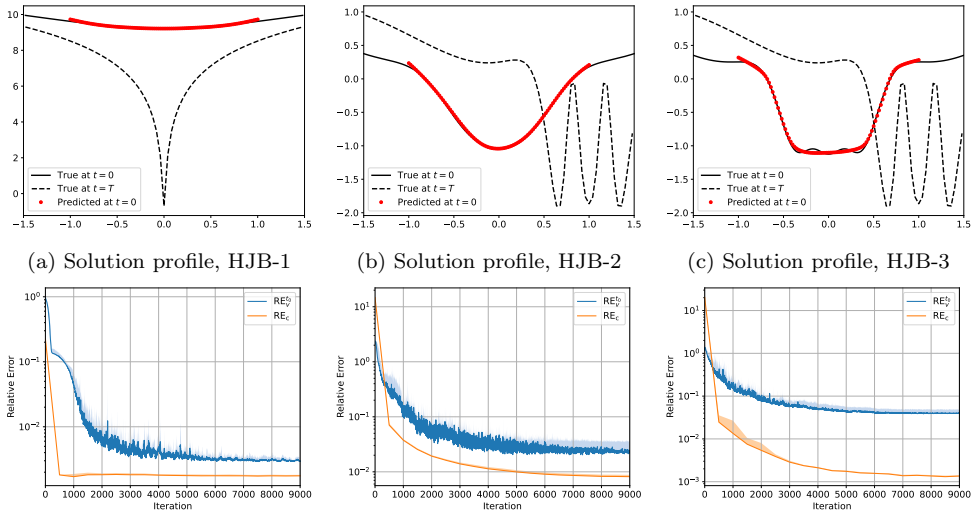
4.2. HJB equations. The considered HJB equations are given by (2.6) with

$$\mathcal{L}^\kappa = (b + 2\kappa)^\top \partial_x + \delta^2 \text{Tr}[\partial_{xx}^2], \quad c(t,x,\kappa) = \delta^{-2}|\kappa|^2, \quad U = \mathbb{R}^d, \quad T = 1, \tag{4.6}$$

where $\delta > 0$, $b \in \mathbb{R}^d$ and the terminal condition $v(T,x) = g(x)$ will be specified later. By the Itô formula, the resulting HJB (2.6) admits an analytic solution given by

$$v(t,x) = -\ln\left(\mathbb{E}\left[\exp\left(-g(x + (T-t)b + \sqrt{2}\delta B_{T-t})\right)\right]\right) \tag{4.7}$$

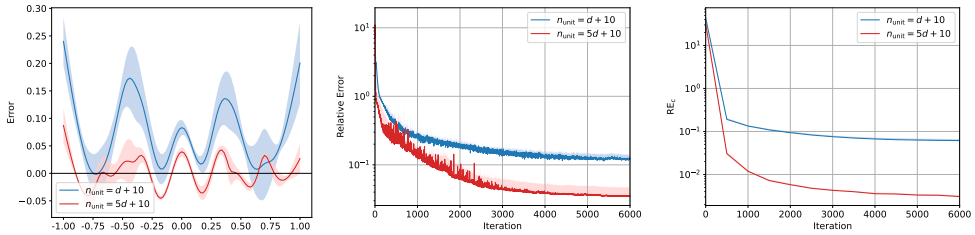
[1] https://github.com/pytorch/tutorials/blob/main/intermediate_source/ddp_tutorial.rst



(a) Solution profile, HJB-1 (b) Solution profile, HJB-2 (c) Solution profile, HJB-3

(d) RE vs Iter., HJB-1 (e) RE vs Iter., HJB-2 (f) RE vs Iter., HJB-3

FIG. 2. *Results for HJB-1, -2, and -3 in subsection* 4.2 *with* $d = 10^4$. *Shading denotes the five-run mean plus or minus twice the SD in (a)–(c) and the five-run mean plus twice the SD in (d)–(f). The final-iteration statistics are reported in Table* 1.



(a) $s \mapsto v_\theta(0, s\mathbf{1}_d) - v(0, s\mathbf{1}_d)$ (b) $\text{RE}_v^{t_0}$ vs Iter. (c) RE_c vs Iter.

FIG. 3. *Results for HJB-3 in subsection* 4.2 *with* $d = 2000$ *and* $n_{\text{unit}} = d+10$ *or* $n_{\text{unit}} = 5d+10$. *Shading denotes the five-run mean plus or minus twice the SD in (a) and the five-run mean plus twice the SD in (b)–(c). The final-iteration statistics are reported in Table* 1.

for $(t, x) \in [0, T) \times \mathbb{R}^d$. For a reference solution, the expectation in (4.7) is approximated by the Monte Carlo method using 10^6 i.i.d. samples of B_{T-t}.

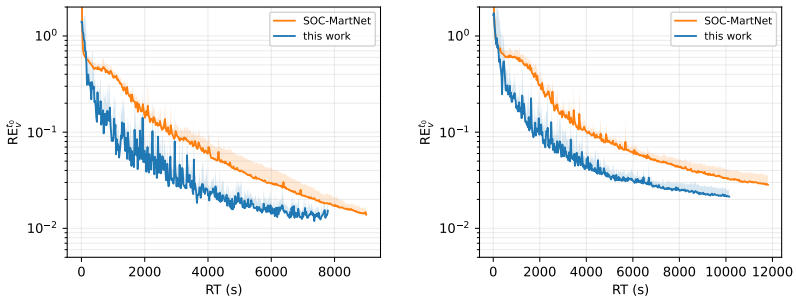
We consider the following specific HJB equations, which are given by (2.6) and (4.6) with different parameters:

- HJB-1: $b = 0$, $\delta = 1$, $v(T, x) = \ln(0.5(1 + |x|^2))$.
- HJB-2: $b = \mathbf{1}_d$, $\delta = 0.2$, and $v(T, x) = \bar{g}(x - \mathbf{1}_d)$ with

$$\bar{g}(x) := \frac{1}{d} \sum_{i=1}^{d} \left\{ \sin(x_i - \frac{\pi}{2}) + \sin((0.3/\pi + x_i^2)^{-1}) \right\}.$$

- HJB-3: a variant of HJB-2 with $\delta = 0.1$.

We have the following comments for the above equations: HJB-1 is a benchmark problem commonly used in existing works, such as [2, 23, 26, 30, 33, 36, 55, 63]. HJB-2 and HJB-3 are more challenging than HJB-1 due to their terminal function $\bar{g}(x)$,



(a) HJB-2 (b) HJB-3

FIG. 4. *Relative errors versus the RT for Algorithm* 3.1 *and SOC-MartNet [16] on HJB-2 and -3 in subsection* 4.2 *with* $d = 10^4$. *The curves and shading denote the mean and the mean plus twice the SD over the independent runs of each method. Device:* $8\times A100$.

which is highly oscillatory around $x = 0$, resulting in an uneven solution when δ is small; see Figure 2(b) and (c). Because b and δ are independent of x, and because of the specific forms of the terminal functions g, it follows from (4.7) that the value functions v of HJB-1, HJB-2, and HJB-3 possess a one-dimensional structure, i.e., $v(t, x)$ of HJB-1 depends on x only through $|x|$, whereas those of HJB-2 and HJB-3 are sums of d identical one-dimensional functions of x_i for $i = 1, 2, \cdots, d$. Although these solutions admit a one-dimensional structure, we adopt this family of HJB equations because they are standard benchmarks in the literature and remain challenging for conventional grid-based PDE solvers at large d. More complex HJB equations with nontrivial state dependence are considered in subsection 4.3.

By applying Algorithm 3.1 to the above HJB equations, the relevant numerical results are presented in Figures 2 and 3 and Table 1. Figure 2 shows that the learned value functions agree well with the reference solutions and that the errors decrease steadily as the number of iterations grows. The relative errors of the value function and of the cost functional in Table 1 are almost all below 5%, indicating that the method learns not only an accurate value function but also a feedback control with near-optimal performance over the explored region of the state space. These results demonstrate that Algorithm 3.1 is effective for HJB equations with state dimension up to 10^4. Moreover, as shown in Figure 3, for HJB-3, whose solution is oscillatory, the accuracy depends on the network width: the wider network attains smaller errors in both the value function and the relative cost. This improvement suggests that network expressivity may become a major accuracy bottleneck for problems with complicated solution profiles.

A comparison with SOC-MartNet [16] on HJB-2 and HJB-3 is presented in Figure 4. For both HJB-2 and HJB-3, Figure 4 shows that the proposed method attains lower values of $\mathrm{RE}_v^{t_0}$ over most of the common range of training wall-clock times. This advantage is consistent with our derivative-free loss formulation in (3.28), whereas SOC-MartNet requires evaluating the gradients of $v_\theta(t, x)$ with respect to x in its loss function.

4.3. SOCP with complex state dynamics. Now we consider a more complex SOCP, in which the components of the state variable X are asymmetrically coupled, to assess the performance of Algorithm 3.1 for problems with intricate dynamics. The problem is an instance of the SOCP (2.4) with the drift and diffusion coefficients

in the state equation (2.1) specified as $\mu(t,x,\kappa) = b + c_0 s\kappa$ and $\sigma(t,x,\kappa) = s$ for $(t,x,\kappa) \in [0,T] \times \mathbb{R}^d \times \mathbb{R}^d$, where b and s are functions of (t,x), defined by

$$b_i = 2\sin\big(t + i + \sum_{k=1}^{d} 0.9^k x_{i+k} - 1\big), \quad s_{ij} = \sqrt{2}c_1 0.9^{|i-j|}\big(1 + 0.5\cos(t + i + x_{i+1})\big)$$

for $i,j = 1,2,\cdots,d$ with $x_{d+k} := x_k$, $c_0 = 2$ and $c_1 = 0.5$. The control set is given by $U = \mathbb{R}^d$. The cost functional is given by (2.3) with

$$c(t,x,\kappa) = \frac{1}{2}|\kappa|^2, \quad g(x) = \ln\Big(1 + \frac{1}{d}\sum_{i=1}^{d} x_i^2\Big) + \frac{0.3}{d}\sum_{i=1}^{d}\sin(0.5x_i) + \frac{2}{d}\sum_{i=1}^{d}\sin x_i. \tag{4.8}$$

By the Cole–Hopf transformation, the value function admits the representation [17, Appendix C]

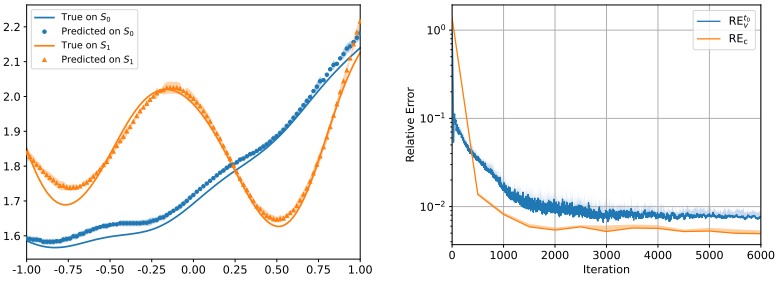
$$v(t,x) = -0.25\ln\mathbb{E}\left[\exp\left(-4g\left(X_T^0\right)\right)\big|X_t^0 = x\right], \quad t \in [0,T), \quad x \in \mathbb{R}^d, \tag{4.9}$$

where X^0 refers to the state process X given by (2.1) with $u \equiv 0$. The reference solution is computed by approximating the expectation in (4.9) via Monte Carlo simulation with 10^6 samples of X_T^0, generated by the Euler–Maruyama discretization of (2.1) with $N = 32$ time steps.

Figure 5 presents the results of Algorithm 3.1 for this SOCP with $d = 10^4$. Despite the asymmetric coupling and nonlinear state dependence, the learned profiles closely track the reference solution on both S_0 and S_1. The final values of $\mathrm{RE}_v^{t_0}$ and RE_c, both below 10^{-2}, indicate that the learned value function is accurate and that the learned feedback control achieves near-optimal performance, respectively.

In the following, we examine the sensitivity of Algorithm 3.1 to the output dimension r of the test function network ρ for this SOCP example. The results are reported in Figure 6. As shown in Figure 6, increasing r from 75 to 1200 reduces $\mathrm{RE}_v^{t_0}$ and RE_c by approximately 35% and 43%, respectively, with diminishing marginal improvement as r increases. Meanwhile, the runtime and GPU-memory ratios vary by less than 1.5% over the tested range. This weak sensitivity is consistent with the shallow architecture of the network ρ_η given in (3.24), whose parameter count grows only linearly with r and, in this configuration, remains much smaller than those of u_α and v_θ. Thus, for Algorithm 3.1, increasing r yields a modest accuracy gain for both the value function and the control at negligible additional computational cost.

Figure 7 reports the per-iteration runtime and GPU memory as the time-step number N, total number M of pilot paths, state dimension d, and mini-batch size $|\mathbb{A}_1| = |\mathbb{A}_2|$ vary one at a time, with all other parameters fixed at their default values. In the first panel, increasing N from 2^3 to 2^7 changes runtime by less than 6% but increases GPU memory by a factor of approximately 2.6, consistent with parallel processing across time levels and path samples. As shown in (3.28) and Algorithm 3.1, increasing M enlarges the retained pilot-path arrays $\hat{X}_n^m$ without changing the mini-batch size $|\mathbb{A}_1|+|\mathbb{A}_2|$, whereas increasing the mini-batch size raises the number of paths processed in parallel without enlarging these arrays. This distinction is consistent with the trends observed in the second and third panels of Figure 7, which show the weak sensitivity of runtime to M and of GPU memory to the mini-batch size $|\mathbb{A}_1| = |\mathbb{A}_2|$. As shown in the fourth panel, the state dimension d is the dominant bottleneck: when d increases from 2^{10} to 2^{14}, runtime and GPU memory increase by factors of approximately 34 and 44, respectively. Both factors are well below the factor



(a) $v(0,x)$ for $x \in S_0 \cup S_1$ (b) $\mathrm{RE}_v^{t_0}$ and RE_c vs Iter.

FIG. 5. *Results for the complex-dynamics SOCP in subsection 4.3 with $d = 10^4$. Shading denotes the five-run mean plus or minus twice the SD in (a) and the five-run mean plus twice the SD in (b).*

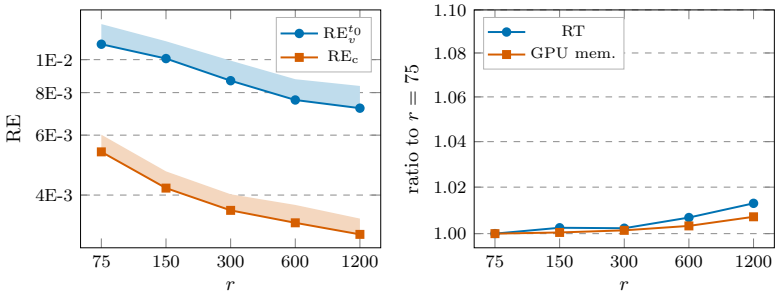


FIG. 6. *Sensitivity to the test-function output dimension r for the SOCP example in subsection 4.3 with $d = 10^4$. Shading denotes the five-run mean plus twice the SD. Device: 4×A100.*

of $2^8 = 256$ implied by quadratic scaling; thus, over the tested range, the observed resource growth is subquadratic rather than exponential and provides no empirical indication of a curse of dimensionality.

4.4. SOCP with a scalar control channel. We consider a SOCP with a scalar control channel to show the effectiveness of Algorithm 3.1 in approximating the optimal control in underactuated problems. The problem is given by (2.4) with the drift and diffusion coefficients in the state equation (2.1) specified as

$$\mu_i(t,x,\kappa) = x_{i+1} - x_i + 2q_i\kappa, \quad \sigma(t,x,\kappa) = \sqrt{2}QQ^\top, \quad (t,x,\kappa) \in [0,T] \times \mathbb{R}^d \times U,$$
$$x_{d+1} := x_1, \quad q := (q_i)_{i=1}^d = 0.5 \times (\sqrt{2},\, \sqrt{2},\, 0, 0, \cdots, 0)^\top \in \mathbb{R}^d, \quad T = 1$$

for $i = 1, 2, \cdots, d$, where $X_0 = \mathbf{0}_d$ and $U = \mathbb{R}$. Here the columns of $Q \in \mathbb{R}^{d\times 20}$ are the first 20 vectors obtained by Gram–Schmidt orthogonalization of the sequence $q, \varphi^{(1)}, \cdots, \varphi^{(d)}$, where $\varphi^{(1)}, \cdots, \varphi^{(d)}$ is the cosine basis of $\mathbb{R}^d$, i.e., the j-th entry of the k-th vector is given by

$$\varphi_j^{(k)} = a_k \cos\big((2j-1)(k-1)\pi/(2d)\big), \quad j,k = 1, 2, \cdots, d,$$

with $a_1 := 1/\sqrt{d}$ and $a_k := \sqrt{2/d}$ for $k \geq 2$. The cost functional is given by (2.3) with

$$c(t,x,\kappa) = \kappa^2, \quad g(x) = 0.5\left(q^\top x - 1\right)^2. \tag{4.10}$$

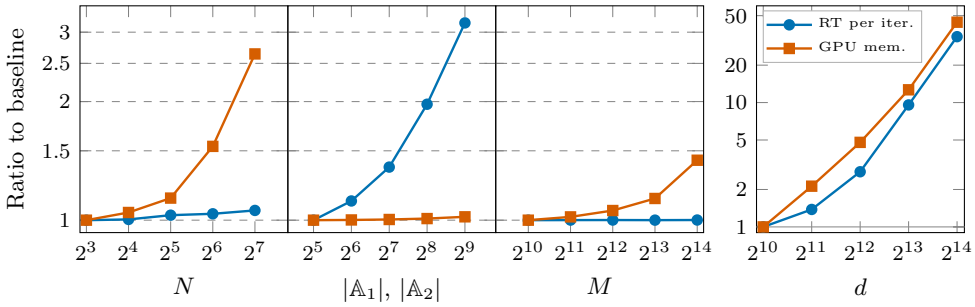
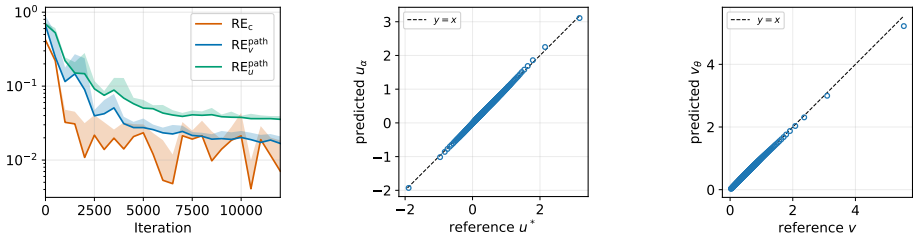


FIG. 7. *Relative per-iteration runtime and GPU memory under parameter sweeps around the configuration of subsection* 4.3 *with* $d = 10^4$. *In each panel, "Ratio to baseline" denotes per-iteration runtime or GPU memory divided by its respective value at the smallest displayed parameter value. Per-iteration values correspond to the outer iteration of Algorithm* 3.1*, indexed by* i. *Device:* $4\times A100$.



(a) REs vs Iter. (b) QQ plot of u_α vs u^* (c) QQ plot of v_θ vs v

FIG. 8. *Results for the SOCP in subsection* 4.4 *with* $d = 10^4$. *In (a), shading denotes the five-run mean plus twice the SD. In (b) and (c), the quantile-quantile (QQ) plots compare* u_α *and* v_θ*, respectively, evaluated on the points* $(t_n, \bar{X}_n^m)$ *introduced in* (4.5).

The optimal feedback control and the value function are given by

$$u^*(t,x) = -q^\top \nabla v(t,x), \quad v(t,x) = \frac{\left(q^\top e^{(T-t)L}x - 1\right)^2}{2A_{T-t}} + \int_0^{T-t} \frac{|q^\top e^{rL} Q|^2}{A_r}\,\mathrm{d}r, \tag{4.11}$$

where $A_r := 1 + 2\int_0^r (q^\top e^{sL^\top} q)^2\,\mathrm{d}s$ and the matrix $L \in \mathbb{R}^{d\times d}$ is such that, for every $x \in \mathbb{R}^d$, the i-th element of Lx equals $x_{i+1} - x_i$ for $i = 1, 2, \cdots, d$, with $x_{d+1} := x_1$.

The results for $d = 10^4$ are shown in Figure 8. The final values of $\mathrm{RE_c}$, $\mathrm{RE}_v^{\mathrm{path}}$, and $\mathrm{RE}_u^{\mathrm{path}}$ are all below 5%. In contrast to evaluating the value function only on the spatial curves S_0 and S_1 at $t = 0$, this example evaluates both v_θ and u_α along the paths driven by the learned control. The close agreement with the diagonal in the two QQ plots further supports the accuracy of both approximations over the path-explored region in high-dimensional state space.

4.5. HJB equation without explicit $\inf_u H$. Now we consider a SOCP whose complexity arises in the Hamiltonian, for which the associated HJB equation reads:

$$\partial_t v + \inf_{\kappa\in\mathbb{R}^d}\left\{(b+\kappa)^\top \partial_x v + \frac{1}{2}|\kappa|^2 + \epsilon\sin(\mathbf{1}_d^\top \kappa)\right\} + \frac{1}{8}\Delta_x v = 0 \tag{4.12}$$

for $(t,x) \in [0,T) \times \mathbb{R}^d$, where ϵ is a scalar parameter specified later, and b is a $\mathbb{R}^d$-valued function of (t,x) given by

$$b_i = \sin(t + i + x_{i+1} - 1), \quad x_{d+1} := x_1, \quad i = 1, 2, \cdots, d,$$

The terminal condition is

$$v(T,x) = g(x) := \ln\left(1 + \frac{1}{d}\sum_{i=1}^{d}\left(x_i^2 + 0.5\sin(10x_i)\right)\right), \quad x \in \mathbb{R}^d, \quad T = 1.$$

This HJB equation arises from the following SOCP:

$$\min_{u \in \mathcal{U}_{\text{ad}}} J(u), \quad J(u) := \int_0^T \mathbb{E}\left[\frac{1}{2}\left|u(t, X_t^u)\right|^2 + \epsilon \sin(\mathbf{1}_d^\top u(t, X_t^u))\right] \mathrm{d}t + \mathbb{E}\left[g(X_T^u)\right],$$

where $X^u : [0,T] \times \Omega \to \mathbb{R}^d$ is the state process defined by

$$X_t^u = X_0^u + \int_0^t (b+u)(s, X_s^u)\,\mathrm{d}s + \int_0^t \frac{1}{2}\,\mathrm{d}B_s, \quad t \in [0,T]. \tag{4.13}$$

The admissible control set $\mathcal{U}_{\text{ad}}$ consists of all Borel measurable functions from $[0,T] \times \mathbb{R}^d$ to $\mathbb{R}^d$, such that the SDE (4.13) admits a unique strong solution.

When $\epsilon = 0$, the value function $v(t,x)$ admits a closed-form solution given by (4.9), where X_t^0 is replaced by the solution of (4.13) with $u \equiv 0$. To compute the reference solution, we approximate the expectation in (4.9) by Monte Carlo, generating 10^6 samples of X_T^0 via the Euler–Maruyama scheme applied to (4.13). For $\epsilon \neq 0$, the infimum in the HJB equation (4.12) does not admit a closed-form solution due to the presence of the sine term. Therefore, this example serves as a test case for evaluating the effectiveness of our method on general problems. In our experiments, we consider a decreasing sequence of the perturbation parameter $\epsilon = 1, 1/2, 1/4, 1/8, 0$ with $d = 10^4$.

As ϵ decreases, the solution of (4.12) is expected to converge to the reference solution at $\epsilon = 0$. The numerical results are presented in Figure 9. As ϵ decreases, the learned value function approaches the reference solution for $\epsilon = 0$, as expected. For each fixed ϵ, the learned value function closely matches the empirical cost over the displayed initial states, indicating that v_θ accurately represents the cost attained by the learned feedback control u_α. These results demonstrate the effectiveness of the proposed method for SOCPs whose Hamiltonian infimization $\inf_u H$ does not admit an explicit closed-form expression.

5. Conclusions. In this paper, we propose a highly parallel, derivative-free martingale deep learning method for high-dimensional SOCPs through their associated HJB equations and the verification theory for the optimal control. The method reformulates the HJB equation into two coupled subproblems: a martingale problem for the value function and an integral optimality condition for the feedback control. As a result, the loss function avoids evaluating the gradient and Hessian of the value function. To enable parallel computation, we introduce a pilot process for state-space exploration and system processes for martingale enforcement, allowing the loss to be evaluated in parallel across both time and space. The martingale condition is imposed in a weak Galerkin form via adversarial learning, which further improves training efficiency and robustness. The optimal feedback control is learned from the integral

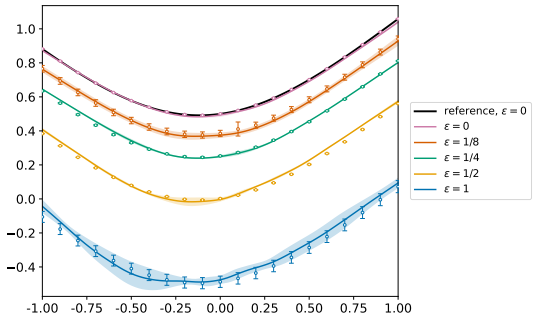


FIG. 9. *Profiles of the learned value function v_θ and of the empirical costs $\hat{J}(u_\alpha)$ for the SOCP in subsection 4.5, with fixed $d = 10^4$ and varying perturbation parameter ϵ. Lines and hollow circles in matching colors respectively depict the mean of $s \mapsto v_\theta(0, s\mathbf{1}_d)$ and of $s \mapsto \hat{J}(u_\alpha)$ defined in (4.5) with $\bar{X}_0^m = s\mathbf{1}_d$, computed over 5 runs. Shading and error bars denote the mean plus or minus twice the SD of v_θ and $\hat{J}(u_\alpha)$, respectively.*

optimality condition, thereby avoiding pointwise minimization for the optimal control. As a result, an efficient training algorithm is proposed to solve the value function and the optimal control simultaneously for high-dimensional SOCPs.

Numerical experiments on a range of high-dimensional problems demonstrate that the proposed method is both accurate and efficient in dimensions up to 10^4, including cases in which the optimal control does not admit an explicit closed-form expression. The results also indicate that network expressivity can become a practical bottleneck for extremely high-dimensional problems with complicated solution profiles. Future work will focus on the theoretical analysis of the proposed scheme, and the development of more efficient network architectures to further enhance the performance of the method for large-scale problems.

Acknowledgments. Authors would like to acknowledge helpful discussions on martingale methods with Xunyu Zhou during the final revision of this manuscript.

Code and data availability. The code reproducing the numerical results in this paper is available at https://github.com/sx-fang/DF-MartNet.

REFERENCES

[1] A. Al-Aradi, A. Correia, G. Jardim, D. d. F. Naiff, and Y. Saporito, *Extensions of the deep Galerkin method*, Appl. Math. Comput., 430 (2022), pp. Paper No. 127287, 18, https://doi.org/10.1016/j.amc.2022.127287.

[2] A. Bachouch, C. Huré, N. Langrené, and H. Pham, *Deep neural networks algorithms for stochastic control problems on finite horizon: numerical applications*, Methodol. Comput. Appl. Probab., 24 (2022), pp. 143–178, https://doi.org/10.1007/s11009-019-09767-9.

[3] G. Barles and E. R. Jakobsen, *On the convergence rate of approximation schemes for Hamilton-Jacobi-Bellman equations*, M2AN Math. Model. Numer. Anal., 36 (2002), pp. 33–54, https://doi.org/10.1051/m2an:2002002.

[4] C. Beck, S. Becker, P. Cheridito, A. Jentzen, and A. Neufeld, *Deep splitting method for parabolic PDEs*, SIAM J. Sci. Comput., 43 (2021), pp. A3135–A3154, https://doi.org/10.1137/19M1297919.

[5] R. Bellman, *On the theory of dynamic programming*, Proc. Natl. Acad. Sci. USA, 38 (1952), pp. 716–719, https://doi.org/10.1073/pnas.38.8.716.

[6] R. Bellman, *Dynamic programming*, Princeton University Press, Princeton, NJ, 1957.

[7] R. Bellman, *Dynamic programming. With a new introduction by Stuart Dreyfus*, Princeton, NJ: Princeton University Press, reprint of the 1957 ed. ed., 2010.

[8] A. Bensoussan, *Stochastic maximum principle for distributed parameter systems*, J. Franklin Inst., 315 (1983), pp. 387–406, https://doi.org/10.1016/0016-0032(83)90059-5.
[9] D. P. Bertsekas, *Dynamic programming and stochastic control*, vol. 125 of Math. Sci. Eng., Elsevier, Amsterdam, 1976.
[10] J.-M. Bismut, *Théorie probabiliste du contrôle des diffusions*, vol. 167 of Mem. Am. Math. Soc., Providence, RI: American Mathematical Society (AMS), 1976, https://doi.org/10.1090/memo/0167.
[11] J.-M. Bismut, *An introductory approach to duality in optimal stochastic control*, SIAM Rev., 20 (1978), pp. 62–78, https://doi.org/10.1137/1020004.
[12] V. G. Boltyanskij, R. V. Gamkrelidze, and L. S. Pontryagin, *The theory of optimal processes. I: The maximum principle*, Transl., Ser. 2, Am. Math. Soc., 18 (1961), pp. 341–382, https://doi.org/10.1090/trans2/018/20.
[13] R. Buckdahn, J. Li, and S. Peng, *Nonlinear stochastic differential games involving a major player and a large number of collectively acting minor agents*, SIAM J. Control Optim., 52 (2014), pp. 451–492, https://doi.org/10.1137/130933174.
[14] S. Cacace, E. Cristiani, M. Falcone, and A. Picarelli, *A patchy dynamic programming scheme for a class of Hamilton-Jacobi-Bellman equations*, SIAM J. Sci. Comput., 34 (2012), pp. A2625–A2649, https://doi.org/10.1137/110841576.
[15] W. Cai, *DeepMartNet – a martingale based deep neural network learning algorithm for eigenvalue/BVP problems and optimal stochastic controls*, July, 2023, https://arxiv.org/abs/arXiv:2307.11942.
[16] W. Cai, S. Fang, and T. Zhou, *SOC-MartNet: A martingale neural network for the hamilton-jacobi-bellman equation without explicit* $\inf_{u\in U} H$ *in stochastic optimal controls*, SIAM J. Sci. Comput., 47 (2025), pp. C795–C819, https://doi.org/10.1137/24M1681033.
[17] W. Cai, S. Fang, and T. Zhou, *Deep random difference method for high-dimensional quasilinear parabolic partial differential equations*, J. Comput. Phys., (2026), p. 114767, https://doi.org/https://doi.org/10.1016/j.jcp.2026.114767.
[18] W. Cai, A. He, and D. Margolis, *DeepMartNet: a martingale-based deep neural network learning method for Dirichlet BVPs and eigenvalue problems of elliptic PDEs in* $\mathbb{R}^d$, SIAM J. Sci. Comput., 48 (2026), pp. c25–c50, https://doi.org/10.1137/24M1630773.
[19] R. Carmona and F. Delarue, *Probabilistic theory of mean field games with applications I. Mean field FBSDEs, control, and games*, vol. 83 of Probab. Theory Stoch. Model., Cham: Springer, 2018, https://doi.org/10.1007/978-3-319-58920-6.
[20] J. Darbon, G. P. Langlois, and T. Meng, *Overcoming the curse of dimensionality for some Hamilton-Jacobi partial differential equations via neural network architectures*, Res. Math. Sci., 7 (2020), pp. Paper No. 20, 50, https://doi.org/10.1007/s40687-020-00215-6.
[21] J. Darbon and T. Meng, *On some neural network architectures that can represent viscosity solutions of certain high dimensional Hamilton-Jacobi partial differential equations*, J. Comput. Phys., 425 (2021), pp. Paper No. 109907, 16, https://doi.org/10.1016/j.jcp.2020.109907.
[22] P. Dupuis, K. Spiliopoulos, and H. Wang, *Importance sampling for multiscale diffusions*, Multiscale Model. Simul., 10 (2012), pp. 1–27, https://doi.org/10.1137/110842545.
[23] W. E, J. Han, and A. Jentzen, *Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations*, Commun. Math. Stat., 5 (2017), pp. 349–380, https://doi.org/10.1007/s40304-017-0117-6.
[24] Y. Fu, W. Zhao, and T. Zhou, *Highly accurate numerical schemes for stochastic optimal control via FBSDEs*, Numer. Math., Theory Methods Appl., 13 (2020), pp. 296–319, https://doi.org/10.4208/nmtma.OA-2019-0137.
[25] B. Gong, W. Liu, T. Tang, W. Zhao, and T. Zhou, *An efficient gradient projection method for stochastic optimal control problems*, SIAM J. Numer. Anal., 55 (2017), pp. 2982–3005, https://doi.org/10.1137/17M1123559.
[26] J. Han, A. Jentzen, and W. E, *Solving high-dimensional partial differential equations using deep learning*, Proceedings of the National Academy of Sciences, 115 (2018), pp. 8505–8510.
[27] J. Han and J. Long, *Convergence of the deep BSDE method for coupled FBSDEs*, Probab. Uncertain. Quant. Risk, 5 (2020), pp. Paper No. 5, 33, https://doi.org/10.1186/s41546-020-00047-w.
[28] J. Han, J. Lu, and M. Zhou, *Solving high-dimensional eigenvalue problems using deep neural networks: a diffusion Monte Carlo like approach*, J. Comput. Phys., 423 (2020), pp. 109792, 13, https://doi.org/10.1016/j.jcp.2020.109792.
[29] J. Han, M. Nica, and A. R. Stinchcombe, *A derivative-free method for solving elliptic partial differential equations with deep neural networks*, J. Comput. Phys., 419 (2020), p. 18, https://doi.org/10.1016/j.jcp.2020.109672.

[30] D. He, S. Li, W. Shi, X. Gao, J. Zhang, J. Bian, L. Wang, and T.-Y. Liu, *Learning physics-informed neural networks without stacked back-propagation*, in Proceedings of The 26th International Conference on Artificial Intelligence and Statistics, F. Ruiz, J. Dy, and J.-W. van de Meent, eds., vol. 206 of Proceedings of Machine Learning Research, PMLR, 25–27 Apr 2023, pp. 3034–3047.

[31] M. Hu, S. Ji, and R. Xu, *A global stochastic maximum principle for forward-backward stochastic control systems with quadratic generators*, SIAM J. Control Optim., 60 (2022), pp. 1791–1818, https://doi.org/10.1137/20M137238X.

[32] Z. Hu, Z. Shi, G. E. Karniadakis, and K. Kawaguchi, *Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks*, Comput. Methods Appl. Mech. Engrg., 424 (2024), pp. Paper No. 116883, 17, https://doi.org/10.1016/j.cma.2024.116883.

[33] Z. Hu, K. Shukla, G. E. Karniadakis, and K. Kawaguchi, *Tackling the curse of dimensionality with physics-informed neural networks*, Neural Networks, 176 (2024), p. 106369, https://doi.org/https://doi.org/10.1016/j.neunet.2024.106369.

[34] Z. Huang, B. Negyesi, and C. W. Oosterlee, *Convergence of the deep BSDE method for stochastic control problems formulated through the stochastic maximum principle*, Math. Comput. Simul., 227 (2025), pp. 553–568, https://doi.org/10.1016/j.matcom.2024.08.002.

[35] C. Huré, H. Pham, A. Bachouch, and N. Langrené, *Deep neural networks algorithms for stochastic control problems on finite horizon: convergence analysis*, SIAM J. Numer. Anal., 59 (2021), pp. 525–557, https://doi.org/10.1137/20M1316640.

[36] S. Ji, S. Peng, Y. Peng, and X. Zhang, *Solving stochastic optimal control problem via stochastic maximum principle with deep learning method*, J. Sci. Comput., 93 (2022), pp. Paper No. 30, 28, https://doi.org/10.1007/s10915-022-01979-5.

[37] S. Ji, S. Peng, Y. Peng, and X. Zhang, *A novel control method for solving high-dimensional Hamiltonian systems through deep neural networks*, SIAM J. Sci. Comput., 47 (2025), pp. c873–c898, https://doi.org/10.1137/24M1647084.

[38] S. Ji and R. Xu, *A stochastic maximum principle for forward-backward stochastic control systems with quadratic generators and sample-wise constraints*, ESAIM, Control Optim. Calc. Var., 30 (2024), p. 25, https://doi.org/10.1051/cocv/2024021.

[39] Y. Jia and X. Y. Zhou, *Policy evaluation and temporal-difference learning in continuous time and space: A martingale approach*, Journal of Machine Learning Research, 23 (2022), pp. 1–55.

[40] Y. Jia and X. Y. Zhou, *Policy gradient and actor-critic learning in continuous time and space: Theory and algorithms*, Journal of Machine Learning Research, 23 (2022), pp. 1–50.

[41] H. J. Kushner, *On the stochastic maximum principle. Fixed time of control*, J. Math. Anal. Appl., 11 (1965), pp. 78–92, https://doi.org/10.1016/0022-247X(65)90070-3.

[42] R. Li, C. Wang, H. Ye, D. He, and L. Wang, *DOF: Accelerating high-order differential operators with forward propagation*, in ICLR 2024 Workshop on AI4DifferentialEquations In Science, 2024, https://openreview.net/forum?id=yQsLKpkRoS.

[43] R. Li, H. Ye, D. Jiang, X. Wen, C. Wang, Z. Li, X. Li, D. He, J. Chen, W. Ren, and L. Wang, *A computational framework for neural network-based variational monte carlo with forward laplacian*, Nat. Mach. Intell., 6 (2024), pp. 209–219, https://doi.org/10.1038/s42256-024-00794-x.

[44] X. Li, D. Verma, and L. Ruthotto, *A neural network approach for stochastic optimal control*, SIAM J. Sci. Comput., 46 (2024), pp. C535–C556, https://doi.org/10.1137/23M155832X.

[45] Z. Liu, W. Cai, and Z.-Q. J. Xu, *Multi-scale deep neural network (MscaleDNN) for solving Poisson-Boltzmann equation in complex domains*, Commun. Comput. Phys., 28 (2020), pp. 1970–2001, https://doi.org/10.4208/cicp.oa-2020-0179.

[46] S. Nadtochiy and T. Zariphopoulou, *An approximation scheme for solution to the optimal investment problem in incomplete markets*, SIAM J. Financ. Math., 4 (2013), pp. 494–538, https://doi.org/10.1137/120869080.

[47] N. Nüsken and L. Richter, *Interpolating between BSDEs and PINNs: deep learning for elliptic and parabolic boundary value problems*, J. Mach. Learn., 2 (2023), pp. 31–64.

[48] B. Ø ksendal, *Stochastic differential equations*, Universitext, Springer-Verlag, Berlin, sixth ed., 2003, https://doi.org/10.1007/978-3-642-14394-6. An introduction with applications.

[49] D. Onken, L. Nurbekyan, X. Li, S. W. Fung, S. Osher, and L. Ruthotto, *A neural network approach for high-dimensional optimal control applied to multiagent path finding*, IEEE Transactions on Control Systems Technology, 31 (2023), pp. 235–251, https://doi.org/10.1109/TCST.2022.3172872.

[50] S. Osher and C. Shu, *High-order essentially nonoscillatory schemes for hamilton–jacobi equations*, SIAM Journal on numerical analysis, 4 (1991), pp. 907–922.

[51] E. Pardoux and S. Peng, *Adapted solution of a backward stochastic differential equation*, Systems Control Lett., 14 (1990), pp. 55–61, https://doi.org/10.1016/0167-6911(90)90082-6.
[52] S. Peng, *A general stochastic maximum principle for optimal control problems*, SIAM J. Control Optim., 28 (1990), pp. 966–979, https://doi.org/10.1137/0328054.
[53] S. Peng and Z. Wu, *Fully coupled forward-backward stochastic differential equations and applications to optimal control*, SIAM J. Control Optim., 37 (1999), pp. 825–843, https://doi.org/10.1137/S0363012996313549.
[54] H. Pham, *Continuous-time stochastic control and optimization with financial applications*, vol. 61 of Stochastic Modelling and Applied Probability, Springer-Verlag, Berlin, 2009, https://doi.org/10.1007/978-3-540-89500-8.
[55] M. Raissi, *Forward-backward stochastic neural networks: Deep learning of high-dimensional partial differential equations*, 2018, https://arxiv.org/abs/arXiv:1804.07010[stat.ML].
[56] M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, J. Comput. Phys., 378 (2019), pp. 686–707, https://doi.org/10.1016/j.jcp.2018.10.045.
[57] S. Richardson and S. Wang, *Numerical solution of Hamilton-Jacobi-Bellman equations by an exponentially fitted finite volume method*, Optimization, 55 (2006), pp. 121–140, https://doi.org/10.1080/02331930500530237.
[58] L. Ruthotto, S. J. Osher, W. Li, L. Nurbekyan, and S. W. Fung, *A machine learning framework for solving high-dimensional mean field game and mean field control problems*, Proceedings of the National Academy of Sciences, 117 (2020), pp. 9183–9193, https://doi.org/10.1073/pnas.1922204117.
[59] Z. Shi, Z. Hu, M. Lin, and K. Kawaguchi, *Stochastic taylor derivative estimator: Efficient amortization for arbitrary differential operators*, in The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024.
[60] I. Smears and E. Süli, *Discontinuous Galerkin finite element approximation of Hamilton-Jacobi-Bellman equations with Cordes coefficients*, SIAM J. Numer. Anal., 52 (2014), pp. 993–1016, https://doi.org/10.1137/130909536.
[61] Y.-H. R. Tsai, L.-T. Cheng, S. Osher, and H.-K. Zhao, *Fast sweeping algorithms for a class of Hamilton-Jacobi equations*, SIAM J. Numer. Anal., 41 (2003), pp. 673–694, https://doi.org/10.1137/S0036142901396533.
[62] E. Vanden-Eijnden and J. Weare, *Rare event simulation of small noise diffusions*, Commun. Pure Appl. Math., 65 (2012), pp. 1770–1803, https://doi.org/10.1002/cpa.21428.
[63] C. Wang, S. Li, D. He, and L. Wang, *Is l^2 physics informed loss always suitable for training physics informed neural network?*, in Advances in Neural Information Processing Systems, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, eds., 2022.
[64] G. Wang, Z. Wu, and J. Xiong, *An introduction to optimal control of FBSDE with incomplete information*, SpringerBriefs Math., Cham: Springer, 2018, https://doi.org/10.1007/978-3-319-79039-8.
[65] X. Wang, W. Zhao, and T. Zhou, *Sinc-θ schemes for backward stochastic differential equations*, SIAM J. Numer. Anal., 60 (2022), pp. 1799–1823, https://doi.org/10.1137/21M1444679.
[66] W. Xu and W. Zhang, *A deep shotgun method for solving high-dimensional parabolic partial differential equations*, J. Sci. Comput., 104 (2025), p. 69, https://doi.org/10.1007/s10915-025-02983-1.
[67] J. Yong and X. Y. Zhou, *Stochastic controls*, vol. 43 of Applications of Mathematics (New York), Springer-Verlag, New York, 1999, https://doi.org/10.1007/978-1-4612-1466-3. Hamiltonian systems and HJB equations.
[68] D. Yoon, K.-W. Jung, E.-T. Jeong, and C.-H. Lee, *Uncertainty-aware adaptive sampling in mppi for robust uav path-following*, Aerospace Science and Technology, 168 (2026), p. 111126, https://doi.org/10.1016/j.ast.2025.111126.
[69] Y. Zang, G. Bao, X. Ye, and H. Zhou, *Weak adversarial networks for high-dimensional partial differential equations*, J. Comput. Phys., 411 (2020), pp. 109409, 14, https://doi.org/10.1016/j.jcp.2020.109409.
[70] T. Zariphopoulou, *A solution approach to valuation with unhedgeable risks*, Finance Stoch., 5 (2001), pp. 61–82, https://doi.org/10.1007/PL00000040.
[71] W. Zhang and W. Cai, *FBSDE based neural network algorithms for high-dimensional quasilinear parabolic PDEs*, J. Comput. Phys., 470 (2022), pp. Paper No. 111557, 14, https://doi.org/10.1016/j.jcp.2022.111557.
[72] W. Zhang, Z. Hu, W. Cai, and G. Karniadakis, *Deep neural networks for solving high-dimensional parabolic partial differential equations*, arXiv:2601.13256. (Jan., 2026).

[73] W. Zhao, L. Chen, and S. Peng, *A new kind of accurate numerical method for backward stochastic differential equations*, SIAM J. Sci. Comput., 28 (2006), pp. 1563–1581, https://doi.org/10.1137/05063341X.
[74] W. Zhao, Y. Fu, and T. Zhou, *New kinds of high-order multistep schemes for coupled forward backward stochastic differential equations*, SIAM J. Sci. Comput., 36 (2014), pp. A1731–A1751, https://doi.org/10.1137/130941274.
[75] W. Zhao, G. Zhang, and L. Ju, *A stable multistep scheme for solving backward stochastic differential equations*, SIAM J. Numer. Anal., 48 (2010), pp. 1369–1394, https://doi.org/10.1137/09076979X.
[76] M. Zhou, J. Han, and J. Lu, *Actor-critic method for high dimensional static Hamilton-Jacobi-Bellman partial differential equations based on neural networks*, SIAM J. Sci. Comput., 43 (2021), pp. A4043–A4066, https://doi.org/10.1137/21M1402303.
[77] M. Zhou and J. Lu, *Solving Time-Continuous Stochastic Optimal Control Problems: Algorithm Design and Convergence Analysis of Actor-Critic Flow*. Preprint, arXiv:2402.17208 [math.OC] (2024), 2024, https://arxiv.org/abs/2402.17208.
[78] M. Zhou and J. Lu, *A policy gradient framework for stochastic optimal control problems with global convergence guarantee*, SIAM J. Control Optim., 63 (2025), pp. 2605–2631, https://doi.org/10.1137/23M1570739.
[79] X. Y. Zhou, J. Yong, and X. Li, *Stochastic verification theorems within the framework of viscosity solutions*, SIAM J. Control Optim., 35 (1997), pp. 243–253, https://doi.org/10.1137/S0363012995279973.